Ravi Natarajan[1]

[1]IBM Corporation, Cognitive & AI

March 05, 2026

Posted on 5 Mar 2026 — CC-BY 4.0 — https://doi.org/10.36227/techrxiv.177274107.74861825/v1 — e-Prints posted on TechRxiv are preliminary reports that are not peer reviewed. They should not b...

K9-AIF: A Governed, Modular Framework for Agentic AI Applications

Ravi Natarajan
IBM Corporation, Cognitive & AI, Atlanta, GA, United States
Email: ravi.natarajan@ibm.com

Abstract

Agentic systems increasingly combine multiple cooperating agents, heterogeneous models, and external tools, creating new challenges in governance, traceability, and architectural extensibility at enterprise scale. Existing orchestration frameworks often emphasize task coordination but provide limited support for consistent policy enforcement, observability, and long-term maintainability.

This article presents a pattern-based architectural approach for governed agentic systems, grounded in a clear separation between Architecture Building Blocks (ABBs) and Solution Building Blocks (SBBs). Configuration-driven factories, runtime agent loaders, and layered architectural abstractions are used to structure orchestration, inference integration, persistence, and monitoring in a modular and governance-first manner. Diagrams illustrate how these elements compose into a cohesive architecture that supports extensibility while preserving stable architectural contracts.

Keywords:

Agentic systems; software architecture; design patterns; AI governance; enterprise AI; modular architectures.

Introduction

K9-AIF is an extensible, agent-based architectural framework that provides reusable building blocks for orchestrating retrieval, enrichment, messaging, storage, security, and monitoring in enterprise environments. It integrates with inference systems such as large language models (LLMs) and vector databases, while emphasizing modularity, governance, and operational scalability.

From an architectural standpoint, agentic systems represent an evolution rather than a departure from established enterprise paradigms. For decades, Service-Oriented Architecture (SOA) and later microservices relied on well-defined contracts, API-based interaction, and centralized orchestration to achieve modularity and governance. These principles remain essential.

What differentiates modern agentic systems is the integration of inference and reasoning into otherwise familiar service flows. Agents delegate knowledge access to **dedicated retrievers**, which obtain domain-specific information from vector databases, APIs, or other external sources through well-defined interfaces. Execution flows are assembled contextually rather than statically, forming goal-driven chains of responsibility. Despite this added intelligence, agents still encapsulate capabilities, expose interfaces, and participate in governed workflows—much like services before them.

K9-AIF adopts the term *agent* to align with current industry terminology, but treats agents as first-class architectural elements subject to the same governance, lifecycle management, and observability expectations as traditional services. This continuity enables agentic systems to integrate cleanly into enterprise ecosystems while benefiting from LLM-powered reasoning.

Vision.

The vision of K9-AIF is to establish a reusable, secure, and governed architectural foundation for enterprise-scale agentic applications. By separating stable **Architecture Building Blocks (ABBs)** from extensible **Solution Building Blocks (SBBs)**, the framework enables abstract design patterns to be realized consistently across domain-specific solutions.

This vision is realized through the following principles:

- **ABB → SBB progression** – a stable architectural backbone extended into domain-specific implementations
- **Configuration-driven extensibility** – LLM providers, vector stores, storage backends, and messaging systems can be swapped without code changes
- **Embedded governance** – authentication, authorization, encryption, secret management, audit trails, and policy enforcement are first-class concerns
- **Operability and observability** – structured logging, metrics, tracing, alerting, and service-level objectives support operational reliability
- **Proof through implementation** – architectural concepts are validated through extensible base classes and reference applications

High Level Process Flow.

The high-level process flow illustrates how requests enter the system through a Web UI or conversational interface and are processed end-to-end by the K9-AIF architecture. Rather than coupling logic directly to infrastructure or tools, each stage of the flow is handled by specialized components that extend stable architectural contracts and participate in a governed execution pipeline.

Context Diagram

This diagram illustrates the high-level interaction between presentation, application, integration, and inference layers, and shows how external tool services and data systems are accessed through governed interfaces.

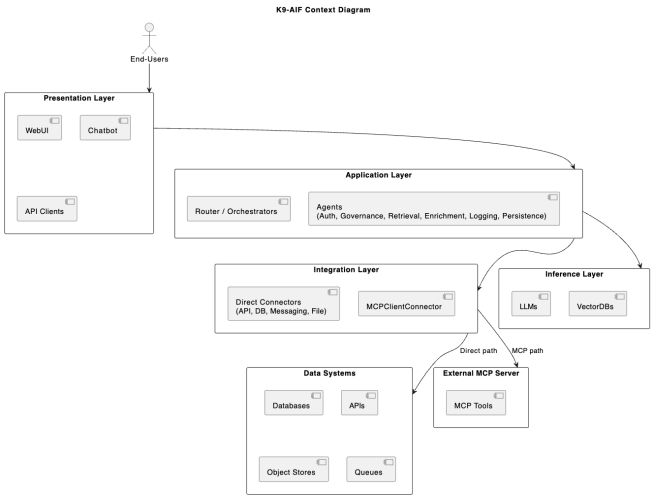

Figure 1. K9-AIF Context Diagram.

Why K9-AIF?

Enterprise adoption of agentic systems requires orchestration mechanisms that are both structured and adaptable. While many agent frameworks emphasize autonomy and task execution, enterprises must also address governance, integration, and long-term maintainability across heterogeneous environments. K9-AIF was created to address these needs by providing a governed, modular architectural foundation for agentic process flows.

Several design drivers informed this approach:

- **Decoupled orchestration** – application logic is separated into reusable orchestrators and agents, enabling clear evolution from architectural contracts to domain-specific realizations
- **Retrieval-augmented generation (RAG)** – retrieval capabilities are treated as first-class architectural concerns, supporting enrichment across structured and unstructured data sources
- **Governance-first design** – security, policy enforcement, and monitoring are embedded consistently across all execution flows
- **Configuration-driven extensibility** – LLM providers, vector stores, storage backends, and messaging systems can be swapped without code changes
- **Proof through implementation** – architectural concepts are validated through working stubs and extensible base implementations

Architectural Foundations.

K9-AIF is grounded in a clear separation between stable architectural contracts and domain-specific realizations. This separation is achieved through the distinction between Architecture Building Blocks (ABBs) and Solution Building Blocks (SBBs).

ABBs define reusable architectural elements—such as agents, orchestrators, and routing components—that establish consistent interaction contracts across applications. SBBs extend these abstractions with domain-specific behavior, enabling concrete solutions to evolve without modifying the underlying architecture.

This separation ensures architectural consistency while allowing solutions to remain relevant to their specific domains.

The architecture is guided by a concise set of principles:

- **Extensibility** – stable base abstractions support polymorphism and runtime binding, allowing ABBs to evolve into domain-specific SBBs
- **Modularity** – layered separation across presentation, application, integration, inference, and data concerns ensures clean boundaries and maintainability
- **Governance-first** – security, policy enforcement, and auditability are applied consistently across all execution flows
- **Interoperability** – heterogeneous databases, APIs, and external connectors are supported through well-defined integration boundaries
- **Observability** – logging, metrics, and tracing provide end-to-end visibility across execution flows

These principles draw from established object-oriented analysis and design practices and align with TOGAF guidance by enforcing separation of concerns, stable contracts, and controlled architectural evolution at enterprise scale.

Architecture.

The K9-AIF architecture is organized into modular layers, each responsible for a distinct set of concerns. This layered structure supports extensibility, separation of responsibilities, and governance-first execution by isolating orchestration, inference, integration, and data access behind stable architectural contracts.

The architecture is presented at two complementary levels. **Figure 1** introduced the system context, illustrating how users, interfaces, and external systems interact with K9-AIF. **Figure 2** refines this view by showing the internal layered organization of the architecture and how responsibilities are distributed across its major components.

The primary architectural layers are:

- **Presentation Layer** – entry point for end-user and system interactions
- **Application Layer** – orchestration and routing capabilities that hosts domain-specific business process flows implemented as SBBs.
- **Integration Layer** – governed access to external systems, tools, APIs, storage, and messaging services
- **Inference Layer** – model invocation, retrieval, enrichment, and reasoning capabilities
- **Data Layer** – persistence, object storage, and queue-based messaging

Cross-cutting concerns such as **security, governance, and observability** are applied consistently across all layers, ensuring authentication, policy enforcement, auditability, and monitoring are enforced end-to-end.

Each layer is defined through **Architecture Building Blocks (ABBs)** that establish stable contracts, which can be specialized into **Solution Building Blocks (SBBs)** to address domain-specific requirements without modifying the core architecture.

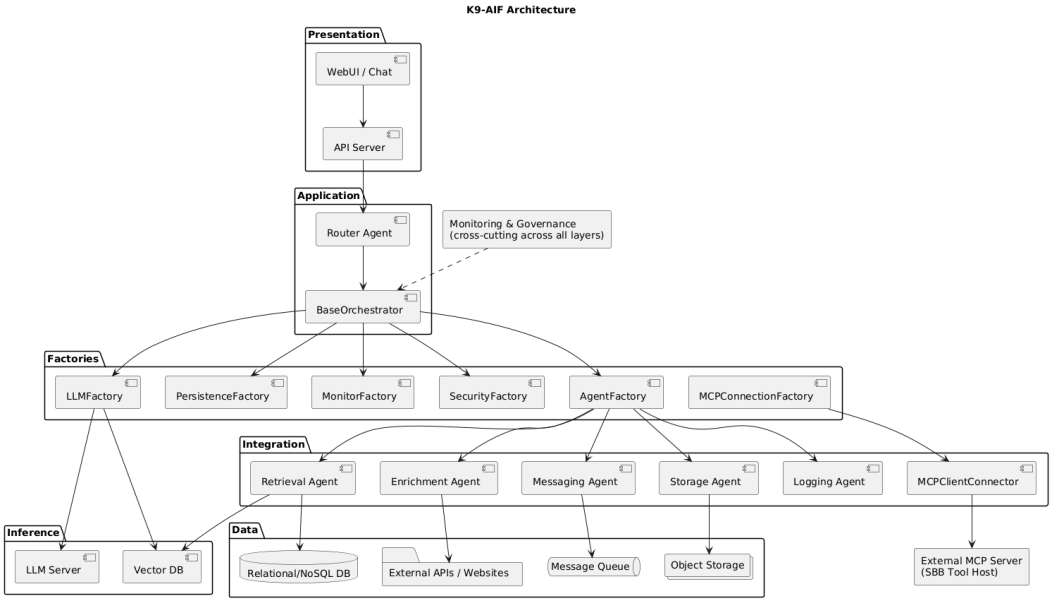

Figure 2. K9-AIF Layered Architecture Diagram.

Detailed Architecture

Business, Data, Applications & Technology (BDAT).

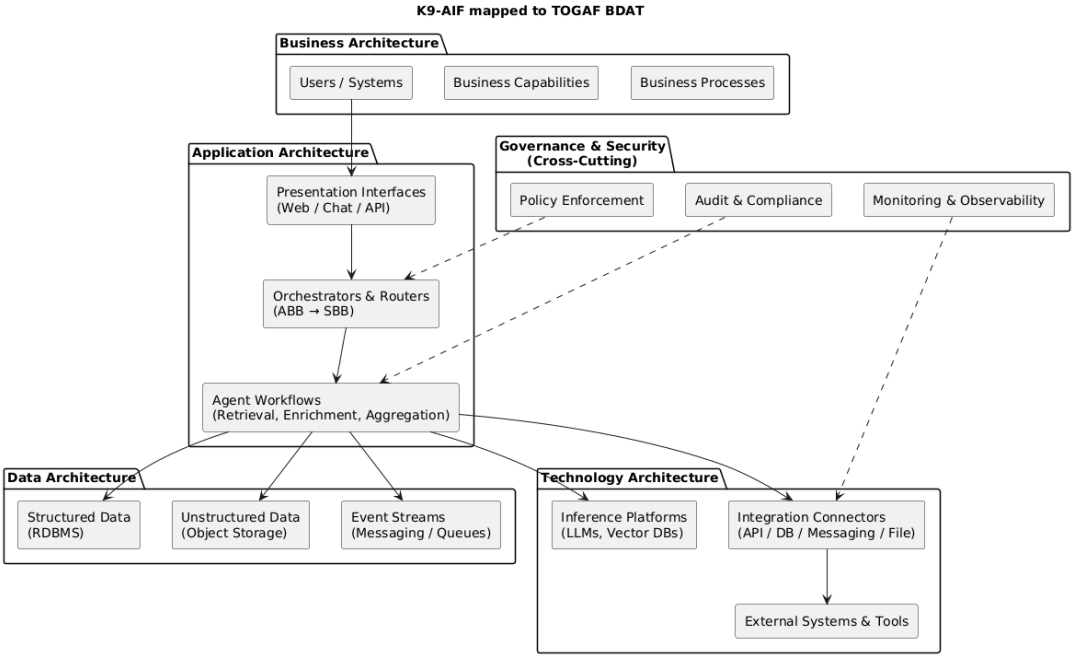

Figure 3. K9-AIF Business, Data, Applications & Technology view.

The BDAT view in Figure 3 provides a classification of architectural concerns across business, data, application, and technology dimensions. The following sections describe how these concerns are realized through a layered runtime architecture, explaining the responsibilities and boundaries of each layer in the order in which requests are processed.

Presentation Layer.

The Presentation Layer serves as the entry point for user and system interactions with K9-AIF. It abstracts interface concerns from orchestration and execution logic, allowing requests to enter the system through multiple channels without impacting downstream processing.

This layer supports web-based interfaces, conversational channels, and API gateways, and is responsible for normalizing inbound requests and formatting outbound responses. Response adaptation is handled through dedicated formatting components, enabling consistent output across formats such as JSON, HTML, or Markdown.

By isolating presentation concerns, the architecture preserves separation of responsibilities and ensures that orchestration flows remain independent of interface-specific details.

Application Layer.

The Application Layer coordinates execution flows by composing agents and services into governed, goal-driven workflows. It is responsible for routing requests, sequencing execution steps, and delegating work to downstream integration and inference capabilities.

Orchestration behavior is defined through reusable architectural contracts that can be extended into domain-specific flows. Domain-specific business logic is introduced only through Solution Building Blocks (SBBs) that extend these contracts; the architectural building blocks themselves remain free of application semantics. This allows generic orchestration logic to remain stable while supporting customization through configuration and specialization rather than framework modification. Different orchestration styles—such as retrieval, enrichment, aggregation, or API-driven flows—can be assembled dynamically to meet application needs.

By isolating orchestration concerns within a dedicated layer, the architecture ensures centralized control of execution while preserving flexibility. This separation allows new workflows to be introduced or existing ones to evolve without impacting presentation logic, integration mechanisms, or inference components.

Integration Layer.

The Integration Layer mediates all interaction between K9-AIF and external systems. It enables agents and orchestrators to access data sources, invoke external services, exchange messages, and interact with storage systems while remaining insulated from integration-specific complexity.

Integration responsibilities are expressed through stable architectural contracts that abstract external systems behind governed interfaces. This allows orchestration and inference logic to remain independent of transport protocols, data formats, and vendor-specific APIs. Integration supports both synchronous and asynchronous interaction patterns, enabling request-response and event-driven flows within the same architectural model.

K9-AIF supports two complementary integration approaches. In the mediated approach, agents interact with external tools and data services through a standardized tool gateway, delegating connector logic outside the core architecture. In the direct approach, agents access external systems through governed connectors when mediation is not required. In both cases, all inbound and outbound interactions are subject to security, policy enforcement, and audit controls.

By isolating integration concerns within a dedicated layer, the architecture enables new systems and services to be introduced without modifying orchestration logic or inference components, preserving extensibility and governance across environments.

Data Layer.

The Data Layer provides persistence and data access services across the K9-AIF framework. It abstracts structured, semi-structured, and unstructured data sources, allowing higher layers to interact with data through consistent architectural contracts without binding to specific storage technologies.

This layer supports a range of persistence needs, including transactional data storage, object and file storage for large artifacts, and messaging systems for asynchronous and event-driven workflows. By isolating data concerns within a dedicated layer, orchestration and inference logic remain independent of backend-specific details. Access to backend databases may occur either through direct persistence adapters or indirectly via APIs exposed by external systems, preserving consistent data access semantics while maintaining integration flexibility.

Data backends are provisioned through configuration-driven mechanisms, enabling storage technologies to be introduced, replaced, or combined without modifying application logic. This approach preserves governance, consistency, and portability across environments while supporting diverse data requirements.

Inference Layer.

The Inference Layer provides the reasoning capabilities of K9-AIF, enabling model invocation, retrieval-augmented generation, and context-aware enrichment. It isolates inference concerns from orchestration and integration, allowing intelligent processing to evolve independently of application flows.

This layer supports interchangeable model providers and semantic retrieval mechanisms, enabling applications to combine large language models with structured and unstructured data sources. Contextual enrichment allows reasoning components to augment requests with retrieved knowledge, generated queries, or inferred structure before results are returned to upstream layers.

By abstracting inference behind stable architectural contracts, the framework allows new models, retrieval strategies, or enrichment techniques to be introduced through configuration rather than code changes, preserving extensibility and governance across deployments.

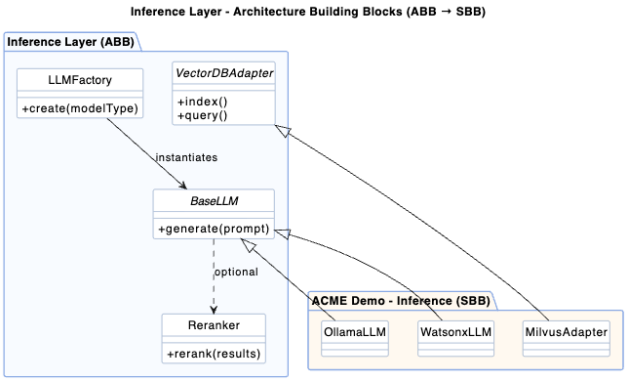

Figure 4. K9-AIF Inference Layer.

Security.

Security in K9-AIF is treated as a cross-cutting concern applied consistently across all architectural layers. Rather than existing as a standalone layer, security capabilities are invoked by agents and orchestrators whenever requests are processed, external systems are accessed, or state is persisted.

The framework provides architectural abstractions for authentication, authorization, encryption, and sensitive data handling. These abstractions allow security mechanisms to be enforced uniformly while remaining adaptable to enterprise identity providers and deployment environments. Policy enforcement and

audit hooks ensure that security controls are applied before and after critical operations, supporting compliance and traceability without embedding security logic directly into application code.

By externalizing security concerns behind stable contracts, K9-AIF enables organizations to evolve authentication methods, authorization policies, and secret management strategies without disrupting orchestration, integration, or inference logic.

Monitoring & Governance.

Monitoring and governance in K9-AIF are treated as cross-cutting concerns applied uniformly across all architectural layers. Rather than operating as isolated subsystems, observability and policy enforcement are embedded into execution flows to ensure transparency, compliance, and operational trustworthiness.

Monitoring capabilities provide visibility into orchestration and agent execution by capturing metrics, traces, and logs across the lifecycle of each request. This enables teams to observe system behavior, diagnose failures, and measure performance without coupling observability mechanisms to application logic. By abstracting monitoring behind stable contracts, the framework allows observability backends to vary by environment while preserving consistent instrumentation across flows.

Governance capabilities enforce policy, security, and compliance checks before and after execution steps. Governance hooks ensure that requests and responses adhere to organizational rules, regulatory requirements, and audit obligations. Treating governance as a cross-cutting function allows policies to evolve independently of orchestration, integration, or inference logic, while ensuring that every execution path remains governed and auditable.

Factories Overview.

Factories in K9-AIF are responsible for provisioning runtime resources across the framework. Each factory corresponds to a core architectural capability and ensures that agents and orchestrators obtain dependencies in a consistent, governed manner. Direct instantiation is intentionally avoided; all runtime components are created through factories to preserve extensibility, observability, and policy enforcement.

Factories provide a stable abstraction layer between architectural contracts and concrete implementations. This design allows inference engines, monitoring backends, persistence mechanisms, routing logic, and security components to be introduced, replaced, or combined through configuration rather than framework modification. As a result, architectural consistency is maintained while enabling flexibility across environments and deployments.

By centralizing instantiation logic, factories also serve as natural enforcement points for governance and lifecycle management. Configuration-driven discovery allows runtime resources to be selected dynamically, ensuring that orchestration flows remain portable across development, staging, and production environments without code changes.

Extensibility & Customization.

K9-AIF is designed to be inherently extensible, allowing organizations to adapt core architectural abstractions into domain-specific solutions without modifying the framework itself. This is achieved through a clear separation between Architecture Building Blocks (ABBs), which define stable contracts, and Solution Building Blocks (SBBs), which provide specialized implementations for particular industries or use cases.

Extensibility is realized through a plug-in model in which new agents, connectors, or adapters are introduced by extending existing ABB contracts. Domain-specific behavior is encapsulated entirely within SBBs, enabling healthcare, finance, defense, or other industry solutions to coexist on the same architectural

foundation. While ABBs remain stable at any given point in time, they are governed architectural contracts that may evolve deliberately as organizations gain experience through new SBB implementations and associated gap analysis, ensuring the framework matures without sacrificing consistency or backward compatibility.

Customization is primarily configuration-driven. Orchestration flows, governance policies, and integration strategies can be tailored through configuration rather than hardcoded logic, allowing deployments to vary across environments and organizational requirements. This approach enables K9-AIF to function as a long-lived architectural framework that evolves alongside enterprise needs while preserving architectural consistency and governance.

Core Architectural Patterns.

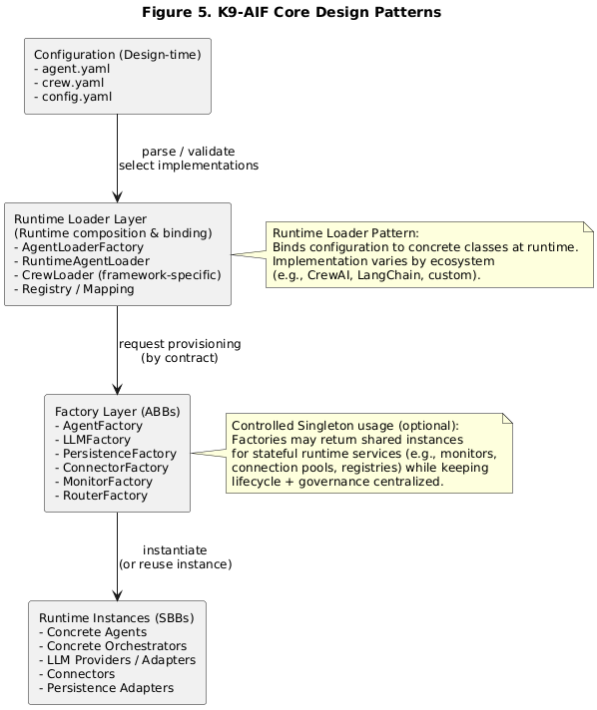

Figure 5. K9-AIF Core Design Patterns.

Factory Pattern.

K9-AIF applies the Factory pattern as the primary mechanism for provisioning runtime components in a consistent and governed manner. Factories encapsulate object creation behind stable architectural contracts, ensuring that agents, connectors, and supporting services are instantiated without direct coupling to concrete implementations. This enables extensibility through configuration and specialization while preserving the stability of core Architecture Building Blocks (ABBs).

This is the point in the architecture where polymorphism and dynamic binding are realized. Concrete implementations are selected at runtime based on configuration and context, allowing multiple SBB implementations to conform to the same ABB contract without framework modification. Centralized instantiation also provides a natural enforcement point for governance, lifecycle control, observability, and policy validation.

In some cases, factories return controlled singleton instances to maintain shared runtime state, while still enforcing lifecycle management and governance through the factory interface.

Runtime Agent loader Pattern.

K9-AIF employs a runtime agent and crew loader pattern to support late binding and configuration-driven composition of agentic workflows. Rather than statically defining agents and orchestration flows, K9-AIF resolves agents and crews dynamically at runtime using declarative configuration.

Agent and crew definitions are expressed externally through configuration artifacts that describe roles, capabilities, and execution relationships independently of concrete implementations. At runtime, a loader component interprets these definitions and instantiates agents through governed factories, selecting the appropriate execution backend based on configuration and environment context.

This pattern decouples architectural intent from implementation choice. The same agent and crew definitions can execute on different orchestration or inference engines without modifying application code. By separating provisioning from composition, K9-AIF enables flexible deployment, environment-specific behavior, and incremental evolution of agentic workflows while preserving architectural stability and governance.

Conclusion.

Agentic systems introduce new flexibility in how software components reason, collaborate, and adapt at runtime, but they also amplify long-standing enterprise concerns around governance, traceability, and architectural stability. K9-AIF demonstrates how established architectural principles—layering, separation of concerns, factory-based provisioning, and runtime composition—can be applied to agentic systems without coupling solutions to specific frameworks or model providers.

By separating stable architectural contracts from domain-specific implementations and resolving agent composition dynamically through configuration, the framework enables governed evolution rather than ad hoc orchestration. While illustrated through concrete examples, the patterns described are broadly applicable to enterprise environments seeking to operationalize agentic AI in a controlled, extensible, and auditable manner.

References

[1] G. Booch, *Object-Oriented Analysis and Design with Applications*. Addison-Wesley, 1991; 2nd ed. 1994.

[2] B. Stroustrup, *The C++ Programming Language.* Addison-Wesley, multiple editions.

[3] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[4] Object Management Group (OMG), *Unified Modeling Language (UML) Specification v2.5.1.* 2017. [Online]. Available: https://www.omg.org/spec/UML/2.5.1

[5] The Open Group, *TOGAF® Standard, 10th Edition*. Van Haren Publishing, 2022.

[6] Anthropic, *Model Context Protocol (MCP)*. 2024. [Online]. Available: https://github.com/modelcontextprotocol

[7] CrewAI, *Multi-Agent Orchestration Framework.* [Online]. Available: https://www.crewai.com

[8] Prometheus Authors, *Prometheus Monitoring System*. [Online]. Available: https://prometheus.io

[9] R. Natarajan, *K9-AIF Framework Documentation*. GitHub, 2025. [Online]. Available: https://github.com/k9aif/k9_aif_abb

Author's Biography

Ravi Natarajan is an Enterprise Architect at IBM with more than 30 years of professional experience in software engineering and enterprise architecture. His background spans systems development in C and C++, object-oriented analysis and design (OOA/OOD), UML modeling, and TOGAF-based architecture practices. He has worked across telecommunications, enterprise integration, and AI-enabled systems, and has held architect roles involving SOA, messaging platforms, and governance-driven frameworks. Ravi holds a Master's degree in Computer Applications (MCA) from PSG College of Technology, India. He is TOGAF 9 certified, an AWS Certified Solutions Architect, and holds multiple IBM certifications. His current interests focus on governed, extensible architectures for agentic and AI-enabled enterprise systems.